

CLAUDE.md — Continuum: Verifiable Continuous Credit Rating on OG

Event: OG Bridge by AKINDO — 10-week, 5-Wave Buildathon **This document scopes:** Wave 3 (4-day build sprint) **Codename:** Continuum (decided — don't re-open this)

0. How to Use This Document

This is a build brief for an agentic coding environment (Claude Code / Hermes), not marketing copy. Hand this whole file to your agent with the kickoff prompt in Section 13 and expect a coherent build-out against a hard 4-day deadline — not the 4-6 week Phase 0 originally scoped for this product.

This is a **rewrite** of the original Continuum product pack, retargeted in two ways:

1. **Infrastructure:** TEE attestation, Chainlink Functions, and generic EVM deployment are replaced with OG's native stack — OG Compute for verifiable inference, OG Storage for the feature store, OG Chain for the score registry.
2. **Scope:** compressed from a 4-6 week off-chain-only Phase 0 into a 4-day build that lands real OG infrastructure by submission — because at Wave 3, mainnet OG Integration Proof isn't just good practice, it's a hard submission requirement.

Sections 1-4 restate product decisions already made — don't re-litigate them. Sections 5-9 are the technical spec and build sequence, written to be executed directly. Section 10 is the submission checklist. Section 13 is the paste-ready kickoff prompt.

1. What We're Building

One sentence: a live credit-scoring layer for RWA lending pools that re-scores individual borrowers continuously from real financial signal instead of once at origination, and publishes that score **verifiably on OG** so pool terms can move automatically as risk changes.

What it is NOT: not a lending protocol (infra/oracle layer other pools plug into), not a general crypto-counterparty risk score, not a KYC/identity product.

Why this fits the hackathon: OG's suggested tracks list **Finance** ("credit for AI agents, on-chain financial intelligence") and **AI Agents** explicitly. Continuum sits natively in

Finance and can cross into AI Agents if the scoring agent itself is tokenized via Agentic ID (Section 5.5, stretch goal). Cross-track projects are called out as favored.

2. Why OG Solves the Hardest Part of This Product

The original product pack's Section 8 ("Trust & Verifiability") was the hardest, most deferred layer: *why should anyone trust that an AI-computed credit score wasn't fabricated?* The original plan was to stand up TEE infrastructure (Automata/Phala/Marlin Oyster) or fall back to Chainlink Functions — meaningful infra work for a small team.

OG Compute Network's verifiable inference is that layer, natively. Instead of building attestation infrastructure from scratch, Continuum's scoring computation runs as an OG Compute job and inherits OG's verifiability guarantees. This is the single biggest reason Continuum is a strong OG fit, not just a project that happens to deploy a contract on OG Chain.

3. Hackathon Scope: Wave 3 (4-Day Build)

Judging weights: Progress & Momentum 40%, OG Integration 30%, Technical Quality & Execution 20%, Traction & Communication 10%. Because this is Wave 3, full mainnet OG Integration Proof — mainnet contract address, an OG Explorer link showing on-chain activity, and clear proof of at least one OG component integrated — is a **hard submission requirement**, not a scoring bonus. The 4-day build has to land on OG Chain mainnet before submission, not testnet.

Wave 3 exit criteria (what "done" means in 4 days)

1. A working scoring pipeline (Sections 5.1-5.2) running against a small synthetic borrower set, producing real Score Publication Payloads.
2. At least one scoring computation actually executed as an **OG Compute** job, with the returned proof/attestation captured and shown in the payload — not mocked.
3. Borrower Feature Records and evidence documents written to **OG Storage**, with the content hash referenced from the published score.
4. A `ContinuumScoreRegistry` contract deployed to **OG Chain mainnet** (required for Wave 3's Integration Proof), with at least a handful of real score-publish transactions and a working OG Explorer link.
5. A React/Vite dashboard showing score history, trigger reason, and OG Explorer / OG Storage links per score — the "verifiability made visible" story is the pitch.
6. Submission artifacts per Section 10.

Explicitly out of scope for Wave 3 (say this openly in the demo, don't hide it)

- XGBoost/LightGBM trained scoring model — no real default data exists yet to train or backtest against. Use the lightweight weighted quant score in Section 5.1 instead, and say plainly this is the placeholder for Phase 2 once real loan-tape data is available.
- Reference lending pool consuming the score to move rate/LTV live — build the pure formula (Section 5.4) but don't wire it to a real pool this wave.
- Multi-operator decentralized scoring consensus — single-operator for Wave 3, same as the original roadmap's honest v1 framing.
- Agentic ID tokenization of the scoring agent — stretch goal only if Days 1-3 finish early (Section 5.5).
- OG Data Availability and OG Pay — not relevant to this product's Wave 3 needs.

4. Product Decisions Already Made (do not re-decide these)

- **Vertical:** invoice financing / trade finance, not general crypto-counterparty risk (that's Credora's lane) and not portfolio/protocol-level risk (Gauntlet/Chaos Labs' lane). Continuum scores individual borrowers/receivables.
- **ICP:** RWA lending protocol as B2B infra buyer — Continuum is infrastructure other pools plug into, not a lending protocol itself.
- **Cooldown rule:** boundary-crossing downgrades always override the cooldown window and publish immediately (one-sided) — a pool should never lend against a stale grade when risk has clearly worsened, even mid-cooldown.
- **Staleness rule:** when a data source goes silent, the score keeps degrading under continued silence rather than plateauing or reversing upward. Silence is treated as worsening information, not neutral information.
- **Attestation field:** the original Phase 0 POC stubbed this with an explicit placeholder type rather than omitting it or faking TEE fields. That placeholder is now filled by the OG Compute attestation object in Section 6.

5. System Architecture — OG-Native

```

[Data Ingestion] -> [Scoring Engine] -> [OG Compute: verifiable inference]
      |                               |
[OG Storage: feature records + evidence]  [OG Chain: ContinuumScoreRegistry]
                                         |
                                         [Dashboard: reads chain + storage]

```

5.1 Data Ingestion + Scoring Engine (off-chain compute, on-chain-bound output)

- Synthetic borrower dataset for Wave 3 — 5-10 invoice-financing borrowers with a few weeks of simulated revenue, repayment, and invoice events, explicitly modeled on real invoice-financing patterns (day-sales-outstanding, payer concentration, on-time repayment rate). Real originator data integration is a later-wave goal, not this wave.
- **Quant score:** a transparent weighted formula over `revenue_trend_90d`, `days_sales_outstanding`, `payer_concentration_top1_pct`, and `on_time_repayment_rate_180d` — not a trained model this wave. Keep it simple and explainable; this is a placeholder, say so in the docs.
- **LLM reasoning agent:** Claude API call producing the structured `llm_flags` object (`covenant_breach`, `adverse_news_detected`, `confidence`, `evidence_refs`) from synthetic covenant/dispute text — this directly reuses existing Claude API / prompt-engineering work from the DeFi digest pipeline.
- **Aggregation:** combine quant score + `llm_flags` into a letter grade (AA-D) + numeric score
 - confidence interval, applying the cooldown and staleness rules from Section 4.

5.2 OG Compute — verifiable inference

- Wrap the aggregation step (at minimum) as an OG Compute job rather than a plain function call. Consult the Compute SDK quick start before designing the call shape: <https://docs.0g.ai/developer-hub/building-on-0g/compute-network/overview>.
- Capture whatever proof/attestation artifact OG Compute returns and store it verbatim in the `attestation` block of the Score Publication Payload (Section 6) — this is the artifact that fills the placeholder the earlier Phase 0 POC left open.
- If the full aggregation step proves awkward to run inside an OG Compute job in the time available, fall back to running just the LLM reasoning call through OG Compute and flag this scope reduction explicitly in the README rather than silently mocking it.

5.3 OG Storage — feature store

- Each Borrower Feature Record (Section 6) and any synthetic evidence documents get written to OG Storage; the on-chain payload carries only the resulting content hash/URI, not the raw record. Reference: <https://docs.0g.ai/developer-hub/building-on-0g/storage/sdk>.
- This is also the honest answer to the original pack's Layer 1 staleness-flag requirement — a borrower whose feed goes quiet is visible as an absence of new OG Storage writes, not just a database flag.

5.4 OG Chain — score registry

- `ContinuumScoreRegistry.sol` : mapping of `borrower_id` -> latest score struct (score, confidence bounds, timestamp, OG Compute attestation reference, OG Storage content hash), updatable only by an authorized scorer address for Wave 3 (single-operator, stated honestly — decentralized multi-operator scoring is explicitly a later-wave roadmap item, not a Wave 3 claim).
- Deploy via Hardhat or Foundry against OG Chain's EVM-compatible RPC — **mainnet**, per Wave 3's Integration Proof requirement; use testnet only for iteration before the final deploy. Quick start: <https://docs.0g.ai/developer-hub/getting-started>; contract deployment guide: <https://docs.0g.ai/developer-hub/building-on-0g/contracts-on-0g/deploy-contracts>.
- Keep the rate/LTV consumption formula (`risk_premium = f(score, confidence_width)`, `max_ltv = g(score)`, +/-50bps **circuit breaker per update**) as a pure function in the repo even if no pool consumes it live this wave — it's what makes the "AI sets risk input, formula sets terms" separation auditable, and it's cheap to include now.

5.5 Agentic ID (ERC-7857) — stretch goal only

If the pipeline above is solid before Day 4, tokenize the Continuum scoring agent itself as an Agentic ID. This is the move that gives the project a genuine cross-track story (Finance + AI Agents) rather than just a Finance submission that happens to run on OG. Don't start this until Section 5.1-5.4 are working end to end — a half-built registry with a working Agentic ID scores worse than a complete registry without one. Reference:

<https://docs.0g.ai/developer-hub/building-on-0g/agentic-id/overview>.

6. Data Schemas

Borrower Feature Record (written to OG Storage)

```
{
  "borrower_id": "brw_01hx...",
  "as_of": "2026-08-28T00:00:00Z",
  "source_freshness": {
    "accounting_feed": "2026-08-27T22:14:00Z",
    "invoice_feed": "2026-08-26T15:02:00Z"
  },
  "features": {
    "revenue_30d": 184200.50,
    "revenue_trend_90d": 0.06,
    "days_sales_outstanding": 41,
    "payer_concentration_top1_pct": 0.34,
    "on_time_repayment_rate_180d": 0.97,
    "days_since_last_late_payment": 63
  },
  "llm_flags": {
    "covenant_breach": false,
    "adverse_news_detected": false,
    "confidence": 0.91,
    "evidence_refs": ["doc_88a...", "doc_88b..."]
  },
  "data_quality_score": 0.94
}
```

Score Publication Payload (published to OG Chain, references OG Storage + OG Compute)

```
{
  "borrower_id": "brw_01hx...",
  "score": "A-",
  "score_numeric": 742,
  "confidence_interval": [710, 768],
  "prior_score": "A",
  "trigger_reason": "scheduled_daily",
  "model_version": "continuum-scoring-v0.1.0-wave3",
  "attestation": {
    "type": "0g-compute",
    "provider": "0g-compute-network",
    "job_id": "...",
    "proof_ref": "0x...",
    "compute_node": "..."
  },
  "storage_ref": {
```

```

    "provider": "0g-storage",
    "root_hash": "0x...",
    "uri": "0g://..."
  },
  "chain_ref": {
    "network": "0g-chain-mainnet",
    "tx_hash": "0x...",
    "contract": "ContinuumScoreRegistry"
  },
  "published_at": "2026-08-28T06:00:03Z",
  "explainability_ref": "explain_9182..."
}

```

7. Smart Contract Spec: ContinuumScoreRegistry.sol

- `struct ScoreRecord { string borrowerId; int16 scoreNumeric; int16 confidenceLow; int16 confidenceHigh; uint64 timestamp; bytes32 computeAttestationRef; bytes32 storageRootHash; string triggerReason; }`
- `mapping(string => ScoreRecord) public latestScore;`
- `function publishScore(...) external onlyAuthorizedScorer` — enforces the cooldown / boundary-crossing-override rule from Section 4 and the +/-50bps circuit breaker from Section 5.4 at the contract level, not just in off-chain logic, so the guarantee is actually on-chain.
- `event ScorePublished(...)` for the dashboard and for judges' Explorer verification.
- Solidity + Foundry, deployed on OG Chain mainnet for Wave 3.

8. Tech Stack

Component	Choice	Why
Data ingestion	Python + Node.js	Matches existing stack
Scoring cadence	GitHub Actions cron	Reuses the exact pattern already operating the DeFi digest pipeline
Quant score	Weighted formula (not trained model)	XGBoost deferred — no default data yet to train/backtest against
LLM	Claude API, structured JSON	Direct reuse of existing prompt-

reasoning	output	engineering work
Verifiable inference	OG Compute Network	Replaces TEE/Chainlink Functions from the original pack
Feature store	OG Storage (+ local Postgres cache for dashboard queries)	Canonical tamper-evident copy on OG; Postgres is just a fast local mirror
On-chain registry	Solidity + Foundry on OG Chain	EVM-compatible, direct deploy
Frontend	React + Vite	Matches existing portfolio site stack
Agentic ID (stretch)	ERC-7857 registration	Cross-track differentiator if time allows

9. Build Sequence (4 Days)

Day 1 — Synthetic borrower dataset + ingestion pipeline -> Borrower Feature Record schema populated for 5-10 borrowers. Quant score formula implemented and sanity-checked.

Day 2 — Claude API reasoning agent producing `llm_flags`. Wire the aggregation step (or at minimum the LLM call) through OG Compute; confirm a real attestation/proof comes back end to end before building anything downstream of it.

Day 3 — `ContinuumScoreRegistry.sol` written, validated on OG Chain testnet, then redeployed to OG Chain **mainnet** before end of day (Wave 3's Integration Proof requires the mainnet address, not testnet). OG Storage writes for feature records live. First real score-publish transactions on mainnet, correctly referencing both the OG Compute attestation and the OG Storage root hash.

Day 4 — Dashboard (score history, trigger reason, confidence interval, links to OG Explorer and OG Storage). README, architecture diagram, demo video (<=3 min), X post. Agentic ID stretch goal only if everything above is already solid.

10. Submission Checklist (Akindo x OG requirements)

- Project name + one-line description (max 30 words)
- Short summary: what it does, what problem it solves, which OG components it uses
- Public GitHub repo with meaningful commits across the build window + README with setup instructions

- OG integration proof (**mandatory at Wave 3**): OG Chain **mainnet** contract address, an OG Explorer link showing the publish transactions, and clear proof of at least one OG component integrated — this is a hard submission requirement at this wave, not just a scoring bonus
- Demo video, max 3 minutes, showing core functionality, user flow, and the OG integration specifically — the score's `attestation / storage_ref / chain_ref` fields are the part worth putting on screen
- Architecture diagram / technical description (Section 5's diagram, cleaned up)
- Local deployment / reproduction steps
- Mandatory public X post: project name, demo screenshot/clip, hashtags `#0GBridge` `#BuildOnOG`, tags `@0G_labs` `@0G_Builders` `@AKINDO_io`
- Optional bonus: pitch deck, testing notes, technical write-up

11. Manipulation Resistance (condensed — state as roadmap, don't over-build this wave)

- Data spoofing: synthetic data this wave, so not a live concern yet — but say in the README that direct-API integration over borrower-uploaded documents is the intended Phase 2 mitigation.
- Oracle-operator risk: single-operator for Wave 3. State this honestly in the docs rather than overclaiming trustlessness — judges and any real institutional reader will find the overclaim faster than it's worth making.
- Gaming the re-score cadence: event-driven + randomized timing is the intended mitigation, not required to fully implement this wave.

12. Open Assumptions to Flag

If the build agent has to guess on any of these, it should say so explicitly rather than silently deciding:

- Whether OG Compute's SDK supports the aggregation step directly or only simpler inference calls — decide the fallback (Section 5.2) as soon as this is known, don't wait until Day 3 to find out.
- OG Chain deployment sequencing — validate `ContinuumScoreRegistry` on testnet first for fast, free-gas iteration, but the Wave 3 submission target is mainnet, not

testnet; budget explicit time in the 4-day window for the testnet-to-mainnet promotion and for re-verifying the first live transactions before the deadline.

- Exact ERC-7857 registration cost/complexity — this determines whether Section 5.5 is actually reachable this wave or slips to Wave 2.

13. Kickoff Prompt

Paste this on its own once the decisions above reflect reality:

Build the Wave 3 scope described in this document for Continuum: a continuous AI credit-scoring engine for the invoice-financing vertical of RWA lending, built on OG. Scope for this session: (1) a synthetic borrower dataset ingestion pipeline normalized into the Borrower Feature Record schema in Section 6, (2) a weighted quant score plus a Claude API reasoning agent producing the `llm_flags` object, (3) a score aggregation step executed as an OG Compute job producing a real attestation, (4) `ContinuumScoreRegistry.sol` validated on OG Chain testnet then deployed to OG Chain **mainnet** (required for Wave 3's Integration Proof), publishing the Score Publication Payload including the OG Compute attestation and OG Storage content hash, (5) a React/Vite dashboard showing score history, trigger reason, and links to OG Explorer / OG Storage per score. Follow the cooldown and staleness rules in Section 4 exactly as specified. Do not build a trained ML model, a live reference lending pool integration, or multi-operator consensus this wave — those are explicitly out of scope per Section 3. Flag any assumption not specified here rather than silently deciding it.